# BRANDGUARD AI

INTELLIGENT BRAND GOVERNANCE

**CodeNexus India 2026 — Round 2 Project Report**

**Team:** WE CODE CODERS

**Prepared:** 5 November 2026

**Live application**
https://brand-guard-ai-platform-dgt1375ui-aditya-tiwari5.vercel.app/

**Source repository**
https://github.com/adityatiwari8603-sys/BrandGuard-AI-Platform

# 1. Executive Summary

BrandGuard AI is a web-based brand-content compliance prototype. It allows users to upload PDF brand guidelines, extract and store guideline text, check draft content against configured rules, inspect explainable findings and recommendations, and download a compliance report. The system is intended to support human reviewers; it does not replace final human approval.

# 2. Problem and Proposed Solution

Organisations publish content through employees, agencies and AI-assisted tools. This can lead to inconsistent product terminology, outdated factual claims and unsupported promotional statements. BrandGuard AI addresses this problem with a guided workflow that compares submitted text with uploaded guidelines and configured compliance rules.

# 3. Implemented Features

| Feature | Implementation status |
|---|---|
| Dashboard and workspace | Implemented |
| PDF guideline upload | Implemented |
| PDF text extraction with PyMuPDF | Implemented |
| JSON-based guideline storage | Implemented |
| Configured terminology checks | Implemented |
| Configured factual-claim checks | Implemented |
| Predefined promotional-claim detection | Implemented |
| Severity, evidence and recommendations | Implemented |
| Downloadable PDF compliance reports | Implemented |
| Human-review notice | Implemented |
| Public frontend and backend deployment | Deployed and tested |

# 4. Technical Architecture

**Frontend:** React, JavaScript, Vite, HTML and CSS.
**Backend:** Python, FastAPI and Uvicorn.
**Document processing:** PyMuPDF.
**Storage:** JSON-based guideline storage.
**Compliance:** Python rule-based validation.
**Reporting:** downloadable PDF compliance reports.
**Deployment:** Vercel frontend and Render backend.

## Workflow

1. User uploads a PDF guideline through the React frontend.
2. FastAPI receives the document and PyMuPDF extracts its text.
3. Guideline metadata and extracted text are stored in JSON.
4. User submits draft content for checking.
5. The compliance service evaluates configured terminology, factual claims and prohibited promotional claims.
6. The API returns structured findings, severity, evidence and recommendations.
7. The frontend displays the results and enables PDF report download.
8. An authorised human reviewer makes the final decision.

### Main backend modules

`backend/app/main.py` — FastAPI application entry point.
`backend/app/services/guideline_parser.py` — PDF text extraction.
`backend/app/services/guideline_storage.py` — guideline persistence.
`backend/app/services/brand_checker.py` — rule-based compliance checks.

## 5. Deployment

| Component | Address |
|---|---|
| Frontend | https://brand-guard-ai-platform-dgt1375ui-aditya-tiwari5.vercel.app/ |
| Backend API | https://brandguard-ai-platform.onrender.com |
| API documentation | https://brandguard-ai-platform.onrender.com/docs |
| Health endpoint | https://brandguard-ai-platform.onrender.com/health |
| GitHub | https://github.com/adityatiwari8603-sys/BrandGuard-AI-Platform |

The latest verified backend deployment corresponds to commit `80a8a80` (Allow production Vercel deployment origin). The frontend was verified using the latest working Vercel deployment URL above. The older stable Vercel alias was observed to return a deployment-not-found 404; therefore, the working deployment-specific URL is supplied here.

## 6. Verification and Test Results

### Positive compliance test

Input: “BrandGuard Cloud helps teams manage their cloud workflows. Our service provides 99.9% uptime.”
**Observed:** No issues detected; total findings 0; human review required. A PDF report was downloaded.

### Negative compliance test

Input included “world's number one solution”, “guarantees 100% success” and “100% uptime”.
**Observed:** 4 findings (3 High, 1 Medium). The checks flagged the configured terminology mismatch, the uptime mismatch against approved 99.9%, and two prohibited promotional claims. Findings displayed evidence, guideline references and recommendations. A PDF report was downloaded.

### PDF parser test

The manual script `tests/test_guideline_parser.py` generated a one-page PDF in memory and successfully extracted the sample text. Observed output: Success=True, Total Pages=1, Total Characters=93. It was run with the repository root in `PYTHONPATH`. This is a manual print-based script, not a pytest assertion-based test suite.

### Frontend production build

The Vite production build completed successfully (2,248 modules transformed). Vite emitted a chunk-size advisory because one minified JavaScript chunk exceeded 500 kB. This is an optimisation warning, not a failed build.

## 7. Evidence Package

The repository evidence folder contains the following verified artefacts:

| Evidence path | Purpose |
|---|---|
| evidence/screenshots/dashboard.png | Application dashboard |
| evidence/screenshots/negative_compliance_test.png | Negative test findings |
| evidence/screenshots/positive_compliance_test.png | Positive test result |

| | |
|---|---|
| evidence/deployment/render_deployment_80a8a80.png | Render deployment status |
| evidence/test-reports/BrandGuard_Negative_Test_Report.pdf | Negative test PDF report |
| evidence/test-reports/BrandGuard_Positive_Test_Report.pdf | Positive test PDF report |

## 8. Scope Changes from Round 1

The initial proposal described broader capabilities such as AI-assisted tone analysis, RAG-based retrieval, visual brand checks, accessibility and collaborative approvals. Round 2 implementation prioritised a functional end-to-end core workflow: PDF upload, extraction, storage, rule-based validation, explainable findings, recommendations and PDF reporting. Advanced capabilities remain planned work and are not represented as completed.

## 9. Known Limitations

- The current engine uses predefined rule-based checks; it is not an LLM- or RAG-based system.
- Contextual tone and brand voice analysis are not implemented.
- Visual checks for logos, colours and typography are not implemented.
- Accessibility analysis and collaborative approvals are not implemented.
- Detection can miss paraphrased or context-dependent violations.
- Scanned PDFs without extractable text are not supported by the current extraction workflow.
- JSON-based data on the current free Render deployment may be ephemeral and is not durable long-term storage.
- The current interface does not provide a verified delete/remove workflow for uploaded guideline records.
- Findings are advisory; human review is required.

## 10. Three Planned Round 3 Improvements

**1. RAG-based intelligent guideline retrieval.** Chunk and embed guideline documents, retrieve relevant passages for each review, and ground findings in precise source references to improve contextual coverage beyond fixed phrase matching.

**2. Visual brand consistency checker.** Analyse marketing creatives for logo usage, approved colours, typography and visual consistency against documented brand rules.

**3. Collaborative approval workflow.** Add roles and permissions, reviewer assignments, comments, approve/reject/change-request actions, revision history and audit logs.

## 11. Team and Contribution

**Team:** WE CODE CODERS
**Project:** BrandGuard AI
**Event:** CodeNexus India 2026
The team submission should be made by the designated team leader. The source repository and evidence folder contain the project implementation and the verification artefacts listed in this report.

Prepared on 5 November 2026. This report describes the implementation and test outcomes verified for the Round 2 submission; planned capabilities and known limitations are explicitly separated from completed functionality.